

Anatomía del Software a Escala

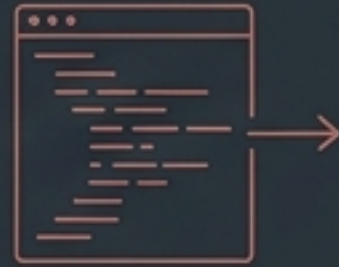
De los cimientos teóricos a las trincheras de la nube.

El Mito (Codificación)

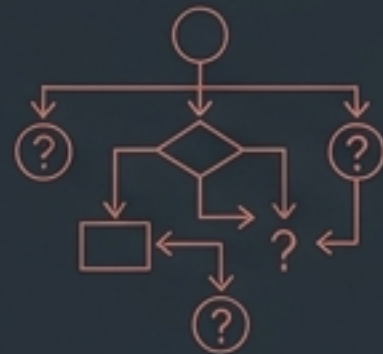
- Programadores aislados en los 70s.



- Foco exclusivo en el código funcional.



- Crecimiento empírico y caótico.



La Realidad (Ingeniería)

- Desarrollo ágil, escalable, mantenible y seguro.

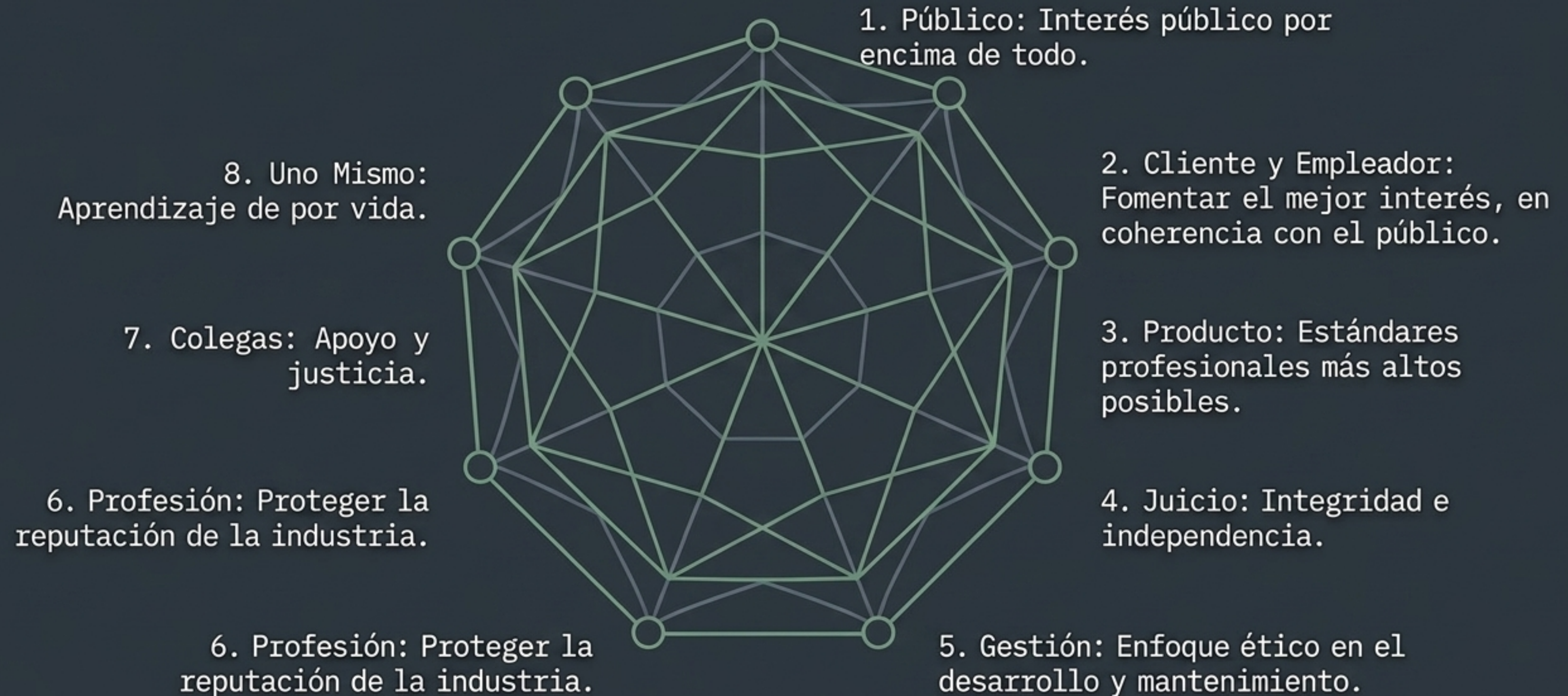


- Tipos de Software: Genérico (población global) vs. A la medida (organizaciones específicas).

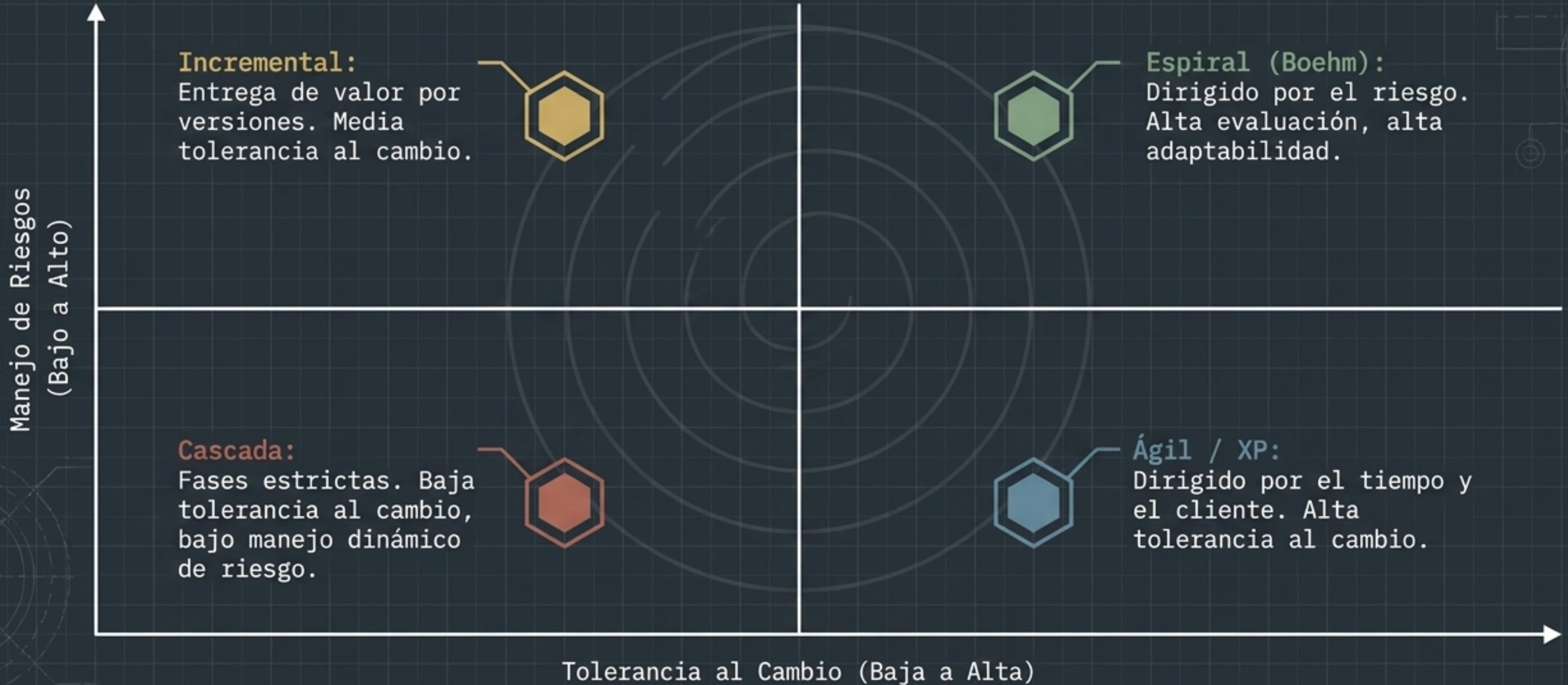


El objetivo real: "Defender el patrimonio de la empresa ante el caos del crecimiento." (Jason Rosso)

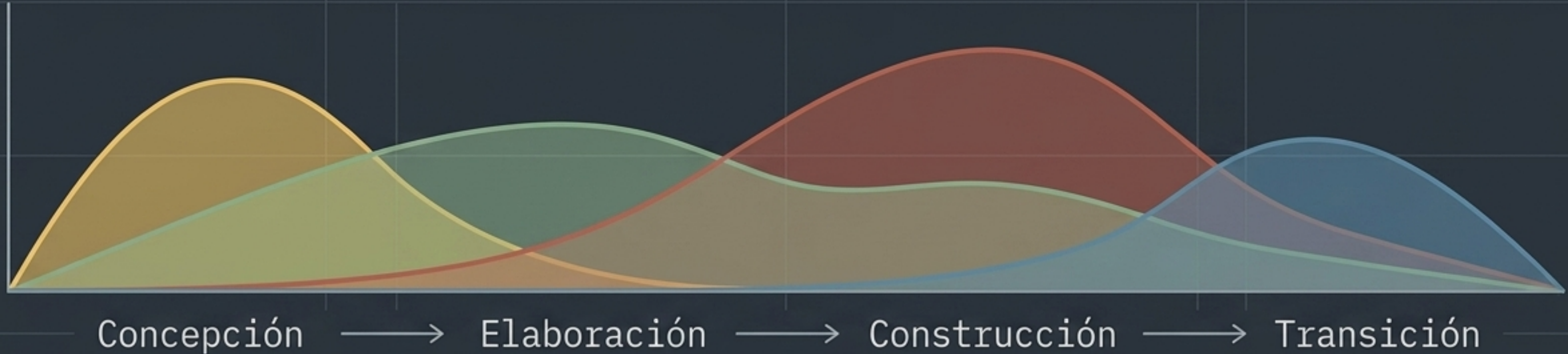
El Núcleo del Profesional: Código ACM/IEEE



De la Anarquía al Orden: Modelos de Proceso



Anatomía del RUP (Rational Unified Process)



Modelado del Negocio: Casos de uso de la empresa.

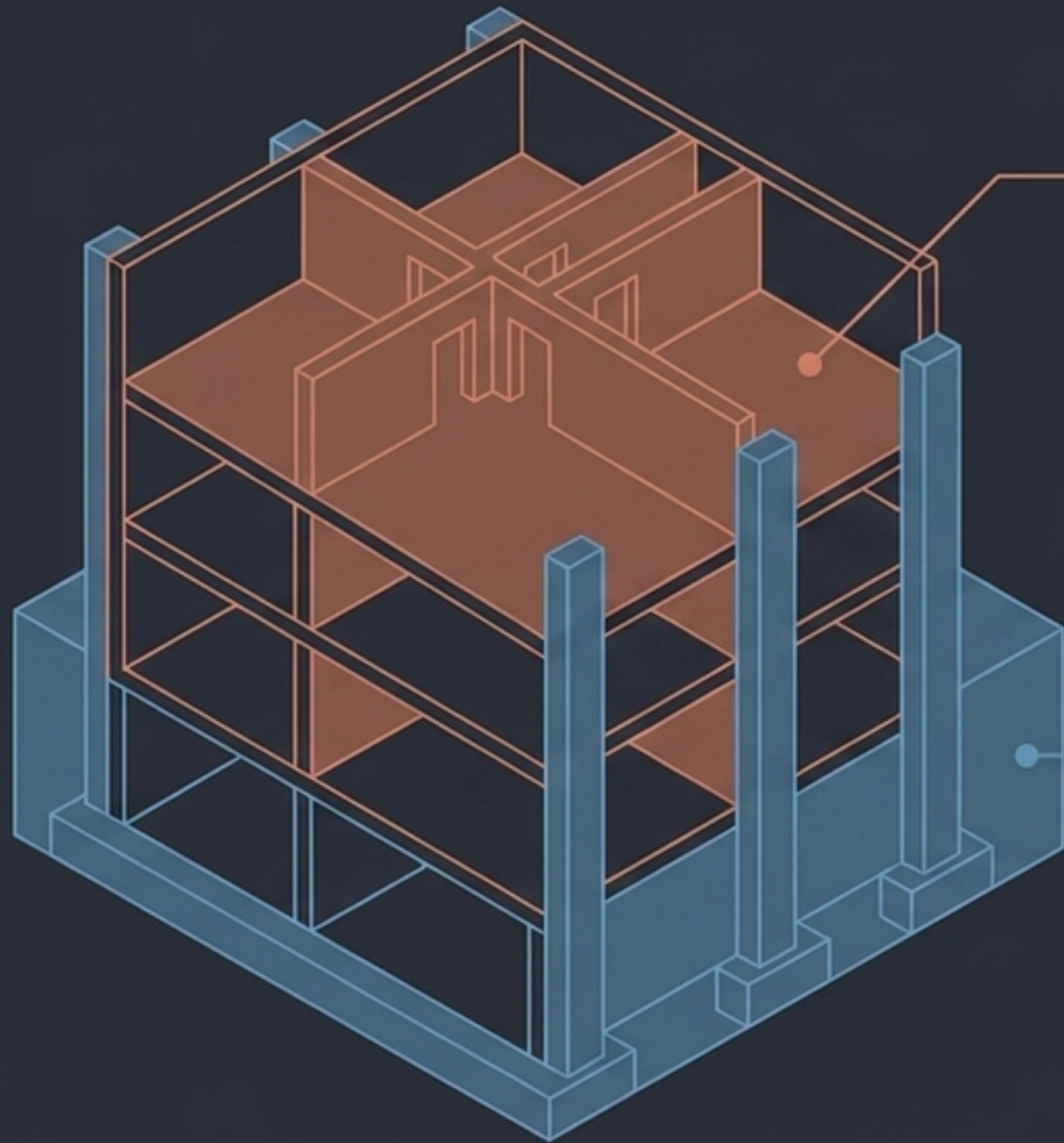
Requerimientos: Identificación de actores.

Análisis y Diseño: Modelos arquitectónicos y UML.

Implementación & Pruebas: Estructuración e iteración.

Despliegue: Liberación e instalación en el entorno.

El Sistema vs. El Ecosistema (Requerimientos)



Funcionales (El Hacer):

Qué debe hacer el sistema.

Ejemplo: "El sistema elaborará informes mensuales de prescripciones."

No Funcionales (El Ser/Restricciones)

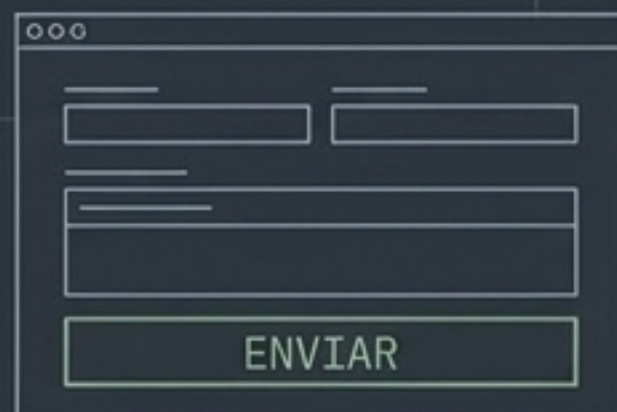
Producto:
Rendimiento,
Usabilidad,
Confiabilidad.

Organización:
Autenticación
con tarjetas de
identidad.

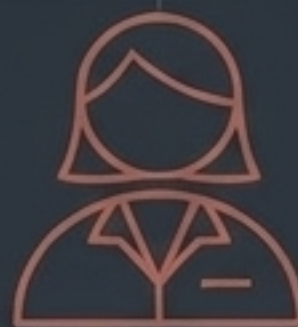
Externos:
Cumplimiento de
privacidad legal
(HStan-03).

El Manejo de Perfiles: Mundos Distintos

La ingeniería de requerimientos fracasa si asume un usuario único.



Operativo: Necesita velocidad, procesos transaccionales y baja fricción (Ej. Recepcionista).



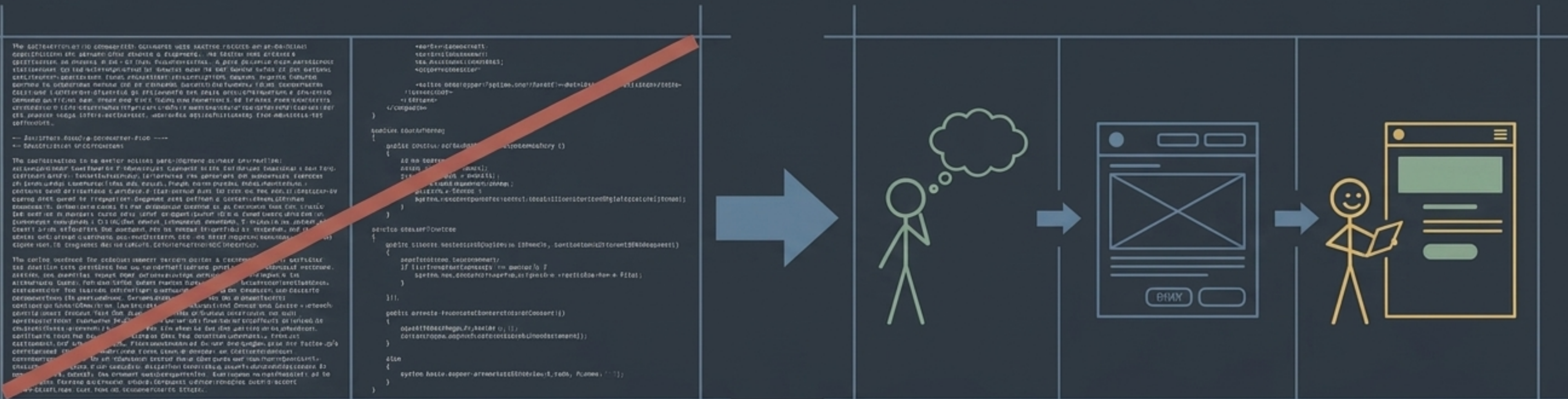
Administrativo: Necesita reportes consolidados, auditoría y control (Ej. Auditor Médico).



Gerencial: Necesita dashboards, visualización de tendencias y apoyo a la toma de decisiones (Ej. Director del Hospital).

Nota de campo: El éxito radica en interfaces limpias pensadas para cada rol.

La "Plastilina Informática" (Pragmatismo UX)



El Problema del SRS: Los clientes rara vez leen o entienden especificaciones de texto denso o diagramas de casos de uso puros.

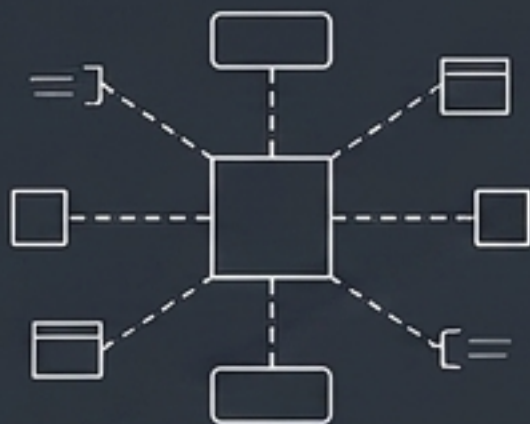
La Solución: Trasladar ideas complejas a gráficos simples, muñecos y maquetas de interfaz.

Insight: "El software debe ser hecho para humanos. A veces la plastilina funciona mejor que el documento de texto para validar requerimientos críticos."

Radiografía del Sistema: Perspectivas UML

Contexto: El Entorno

Límites del sistema y dependencias externas.



Estructural: La Organización

Componentes estáticos, clases, atributos y relaciones de herencia.



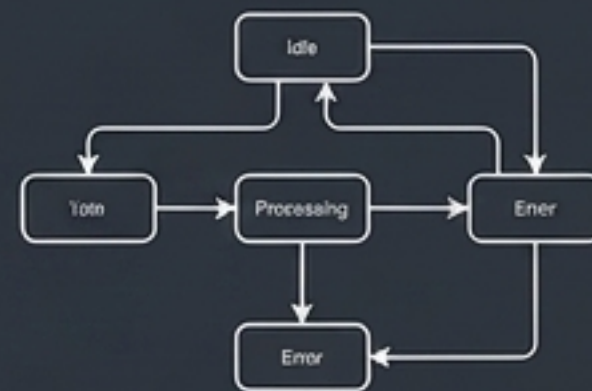
Interacción: El Diálogo

Flujo de mensajes entre actores y el sistema.



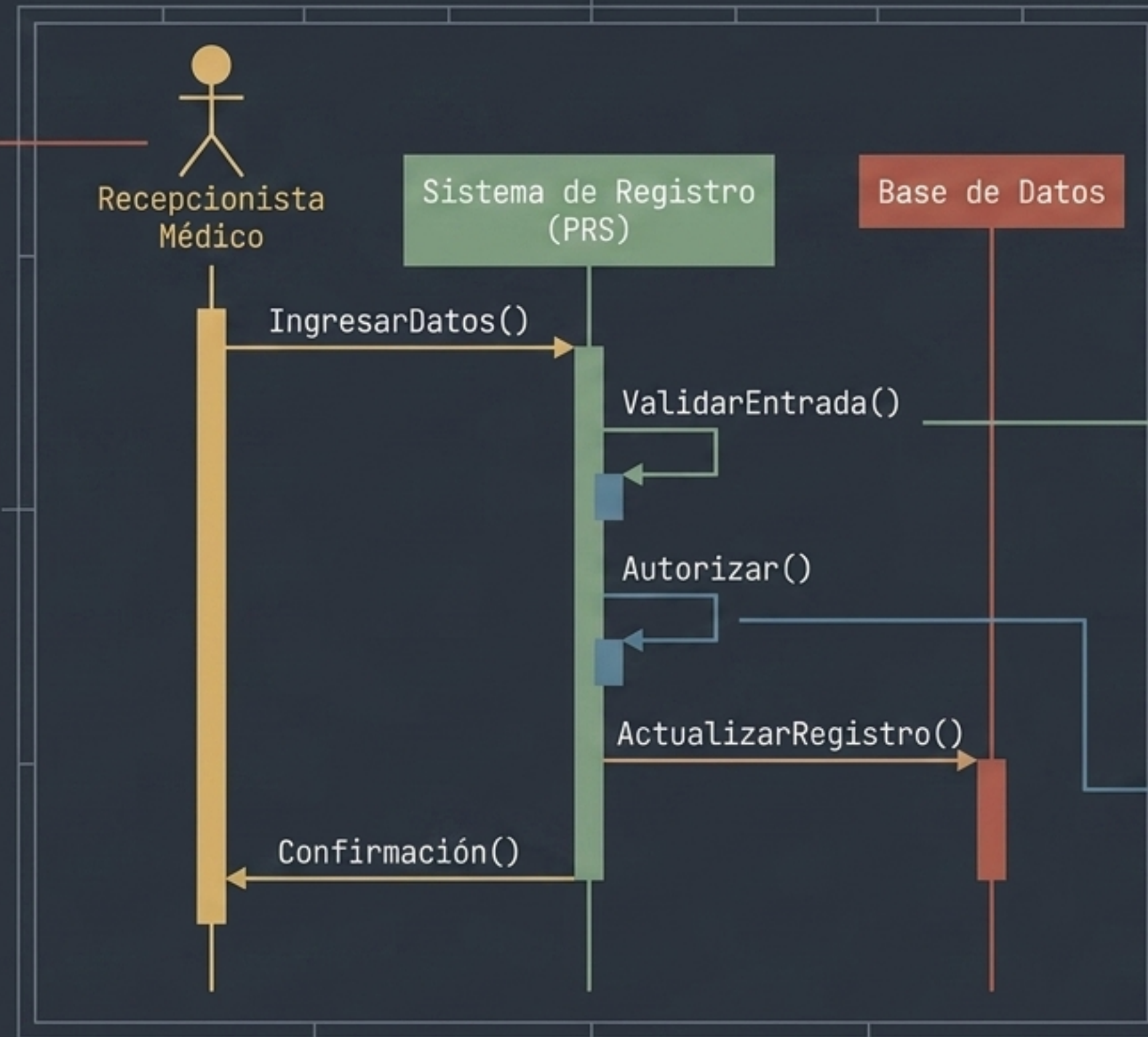
Comportamiento: El Dinamismo

Cómo responde el sistema a eventos internos/externos.



UML en Acción: Diagrama de Secuencia

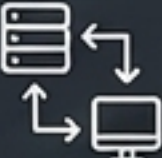
Actores: Recepcionista Médico interactuando con el Sistema de Registro (PRS).



Mensajes: Flujo temporal mostrando validación secuencial.

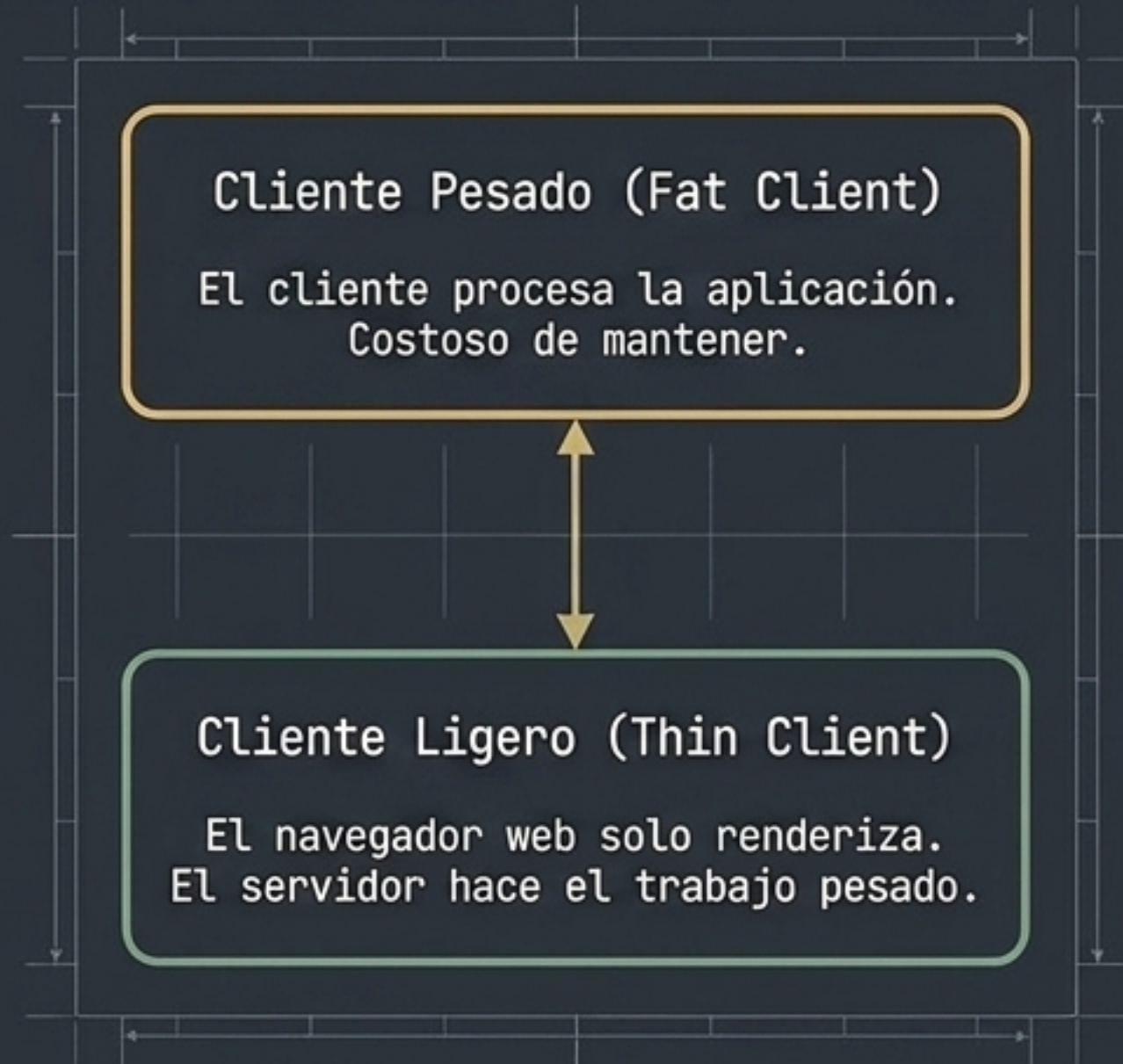
Autorización: Componentes internos verificando permisos antes de impactar la base de datos.

Los Cimientos: Patrones Arquitectónicos

	Patrón	Separación (SoC)	Rendimiento / Despliegue	Escenario Ideal
	MVC	Separa vista, lógica y datos	Múltiples vistas simultáneas	Aplicaciones Web
	Capas	Dependencia estricta top-down	Mantenimiento alto, posible cuello de botella	Sistemas empresariales modulares
	Repositorio	Datos centralizados compartidos	Riesgo de punto único	IDEs, sistemas de información masiva
	Cliente-Servidor	Distribución de carga	Escalable mediante múltiples nodos	Banca en línea, sistemas distribuidos

El Salto a la Escala: Modelos Multinivel (N-Tiers)

Arquitectura 2-Tiers (Cliente-Servidor)

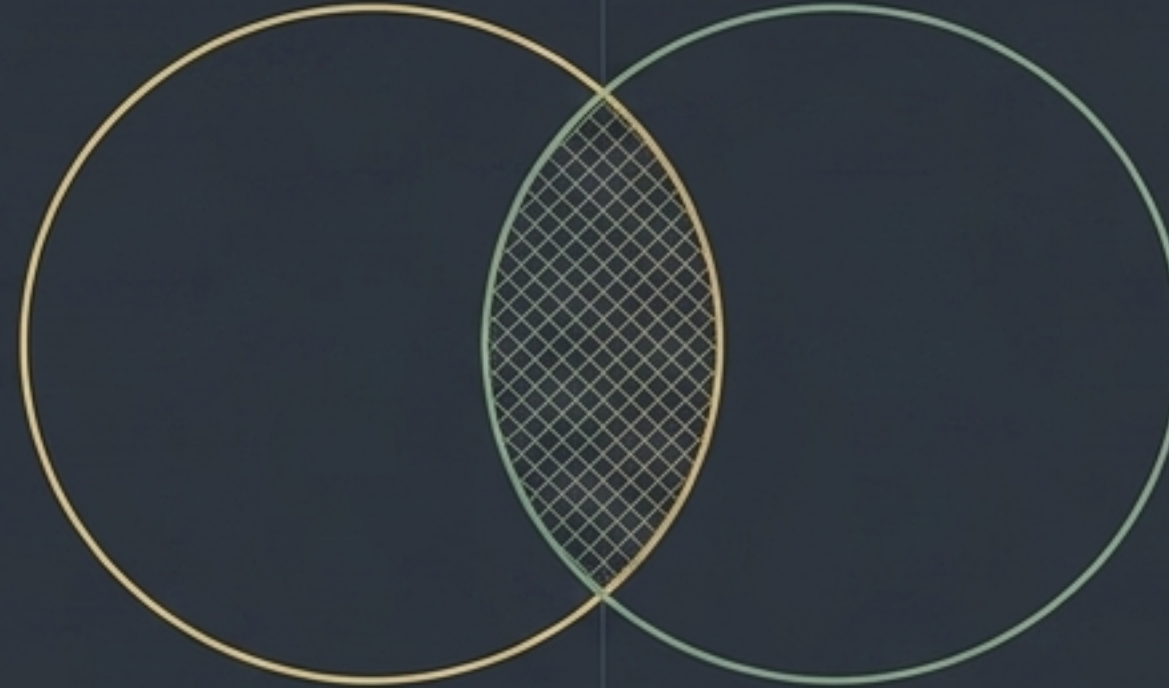


Arquitectura 3-Tiers/N-Tiers:

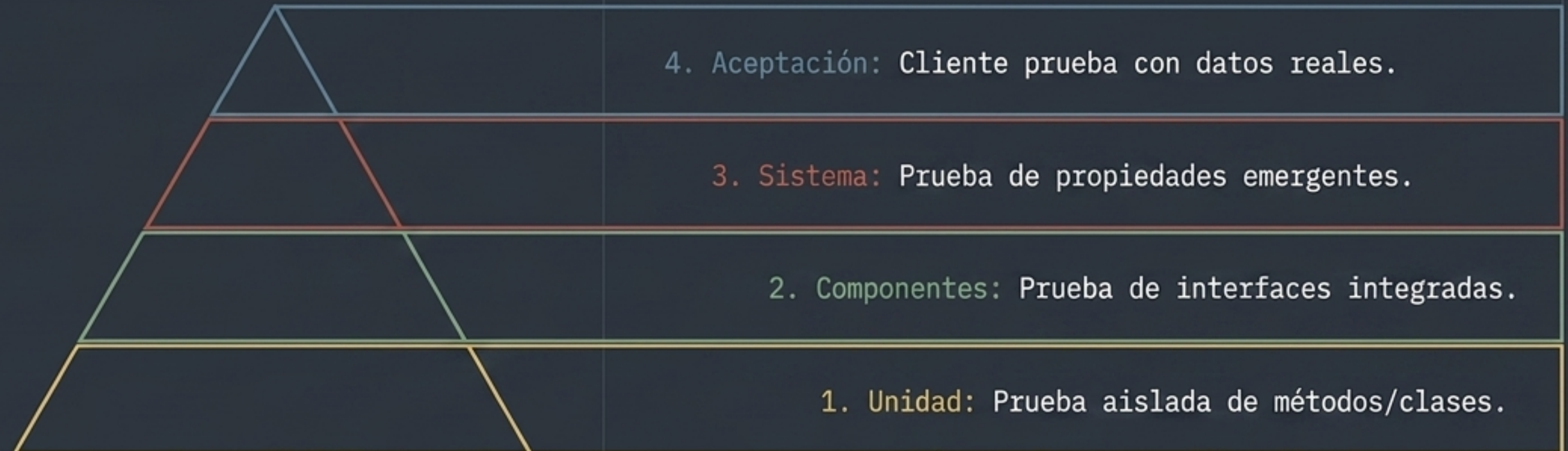


Garantizando la Integridad: Pruebas y Calidad

Verificación: ¿Construimos correctamente el sistema?
(Cumplimiento técnico).



Validación: ¿Construimos el sistema correcto?
(Cumplimiento de la necesidad).



Paradigma Cultural: El Ciclo TDD

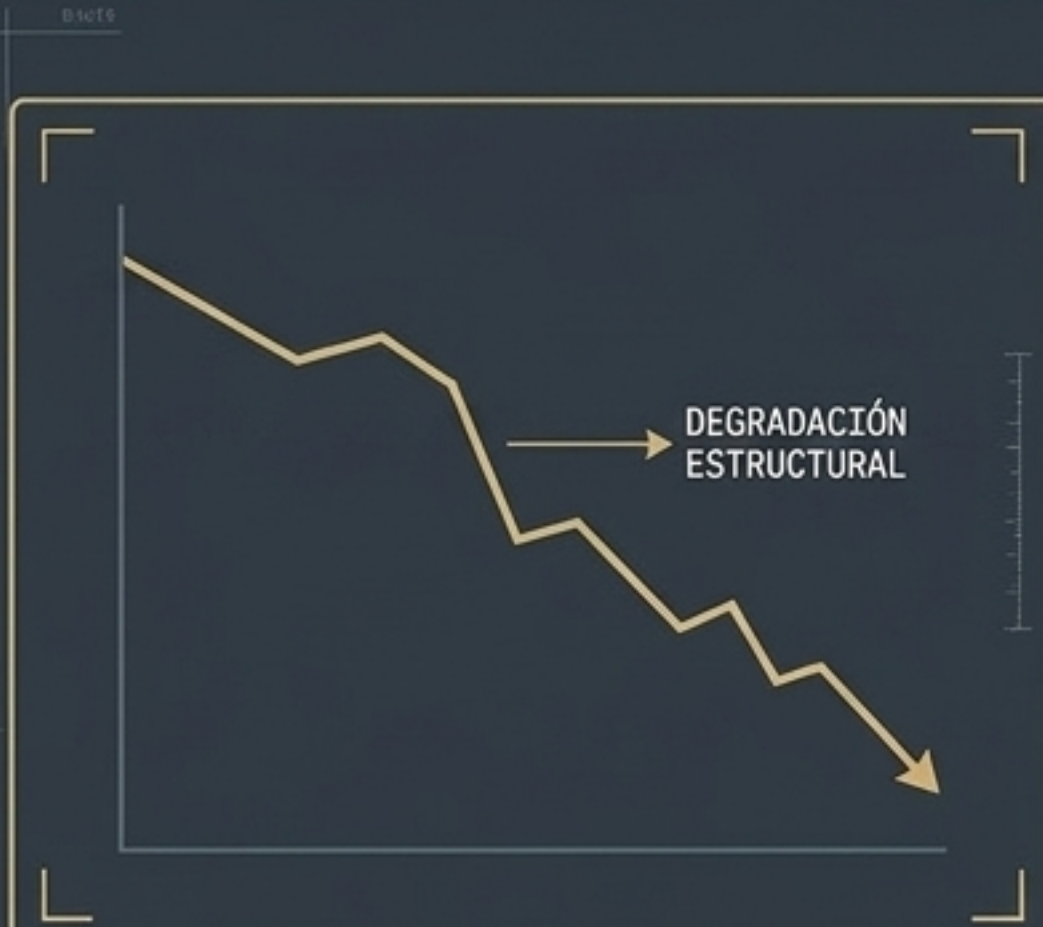
Escribir la Prueba (Fallo):
Definir el comportamiento esperado antes de programar la lógica.

Escribir el Código (Pasa):
Codificar lo mínimo y estrictamente necesario para que la prueba pase.

Refactorizar:
Limpiar el código, optimizar arquitectura, mantener la prueba en verde.

Beneficio: El software se documenta a sí mismo y el desarrollo es dirigido por la fiabilidad.

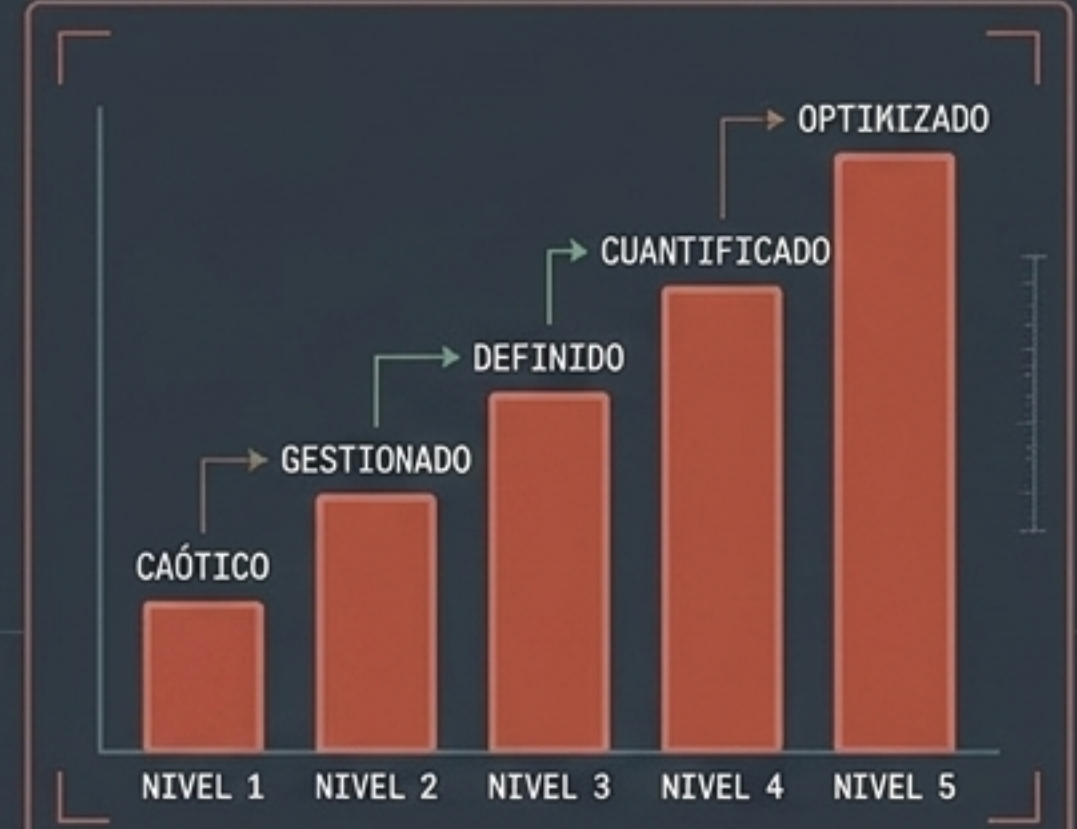
Evolución y Gestión: El Software como Ente Vivo



Leyes de Lehman: El software debe cambiar continuamente o volverse progresivamente inútil. Su estructura se degrada a menos que se invierta en mantenimiento.



COCOMO II: Estimación algorítmica de costos basada en tamaño, complejidad y experiencia del equipo.

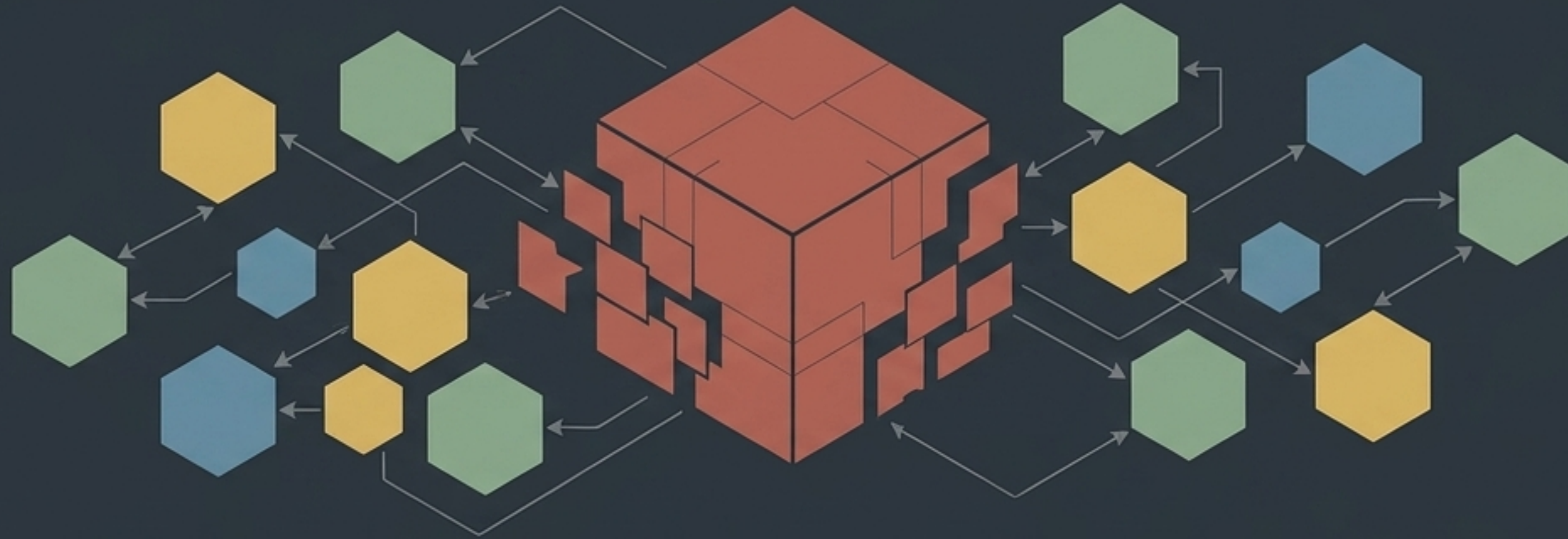


CMMI: Modelos de madurez. De procesos caóticos (Nivel 1) a procesos optimizados y medibles (Nivel 5).

Paradigmas Modernos de Integración

Engineering Blueprint

Editorial Terminal



CBSE (Component-Based SE)

Ensamblaje de componentes estandarizados y probados.



SOA (Service-Oriented Architecture)

Servicios web independientes comunicados por red.



SaaS (Software as a Service)

Despliegue centralizado en la nube; el cliente consume, no instala.

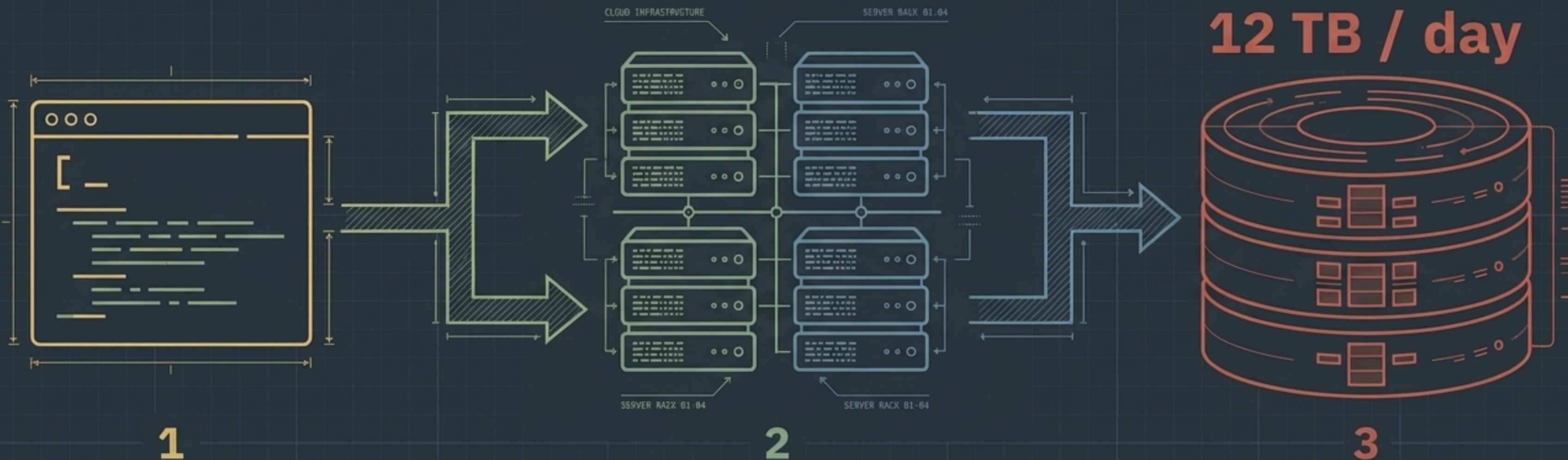


AOSE (Agent-Oriented)

Agentes autónomos para sistemas de alta complejidad.



La Síntesis: Infraestructura a Nivel de Producción



El Proceso: GitHub y metodologías ágiles permiten despliegues continuos.

La Arquitectura: Contenedores, n-tiers, y microservicios desplegados en infraestructuras Cloud.

El Mundo Real: El Reto de Jason Rosso. Monitorizar e ingestar 12 Terabytes de logs.

La ingeniería de software formal existe precisamente para soportar el caos, el riesgo y la escala masiva del mundo real.